

Design Document

HLD · LLD · Architecture & Data Model · Scale. Microservices
on PostgreSQL · offline-first · India-sovereign · security &
observability from day one.

C4 + sequence + ERD

db-per-service + RLS

sharding + partitioning

anti-kickback compliant

Principles & non-functional targets



Right-sized microservices

~12 DDD bounded contexts, hexagonal per service, database-per-service.
No shared DB; events + sagas, not synchronous chains.



Multi-tenant by RLS

tenant_id + Postgres Row-Level Security; claims propagate gateway → service → DB. Large tenants isolatable.



Offline-first edge

Branch nodes keep the counter running through outages; conflict-aware, resumable sync.

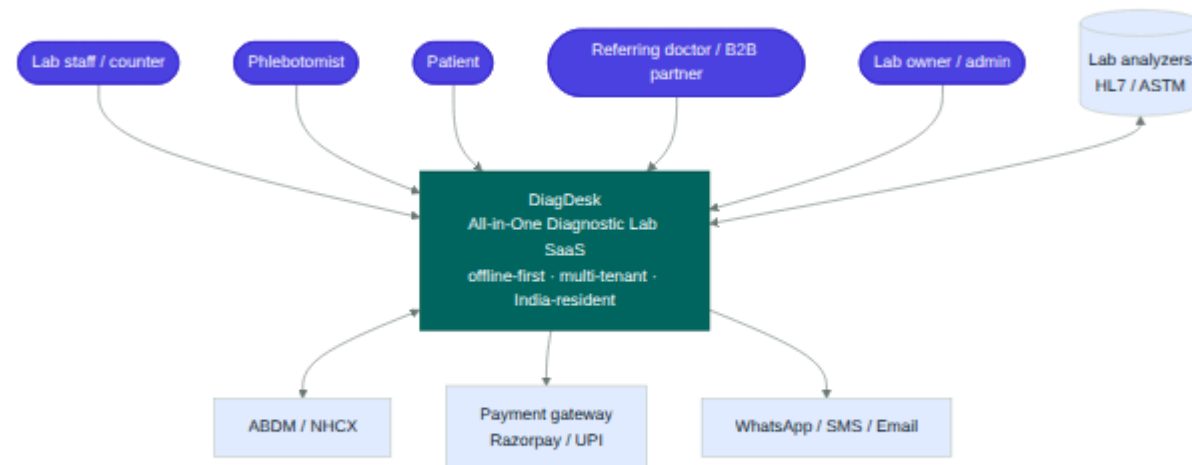


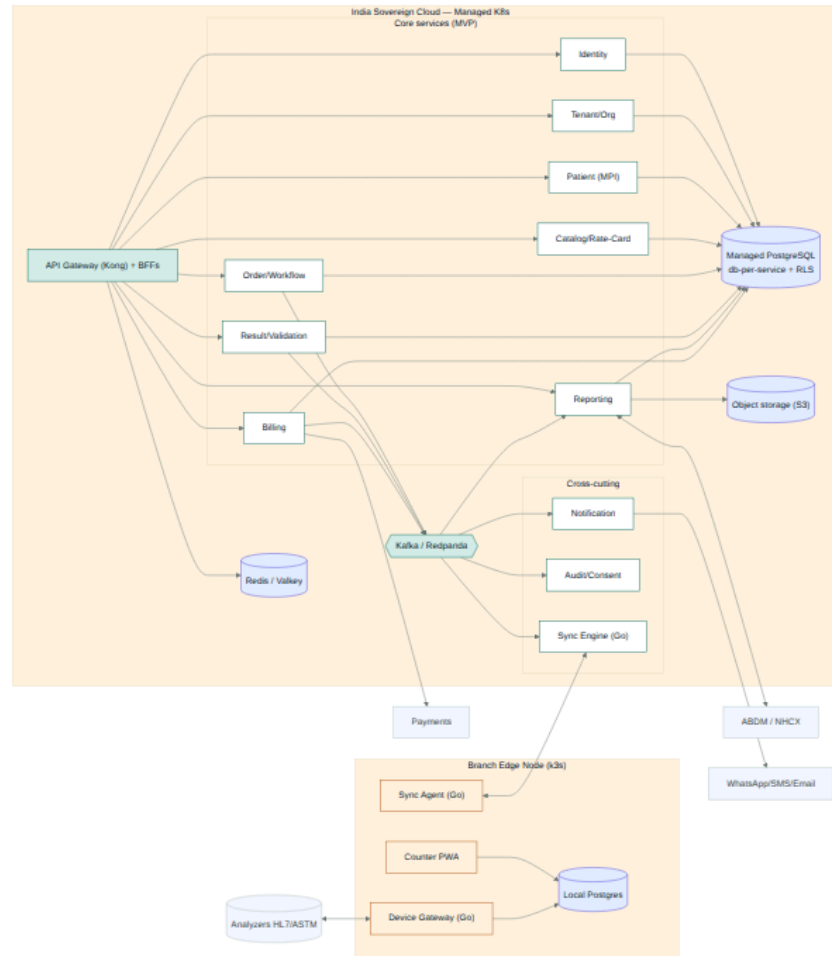
Scales horizontally

Sharding (tenant_id) × partitioning (time) × replicas/CQRS — to millions of records & high txn/min.

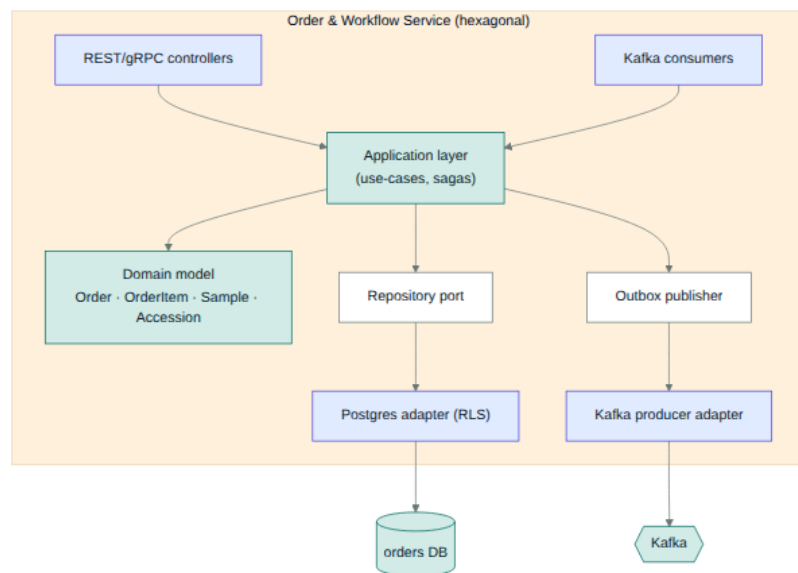
NFR targets: counter ops < 1s p95 (offline) · cloud OLTP write < 50ms p95 · MIS < 500ms p95 · DPDP + CERT-In residency · SLOs on the four golden journeys · **no referral-commission tooling** (ADR-007).

System Context

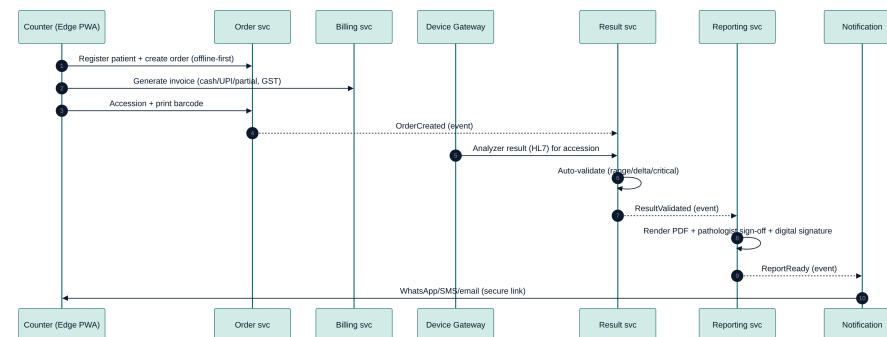




Service internals · Walk-in journey

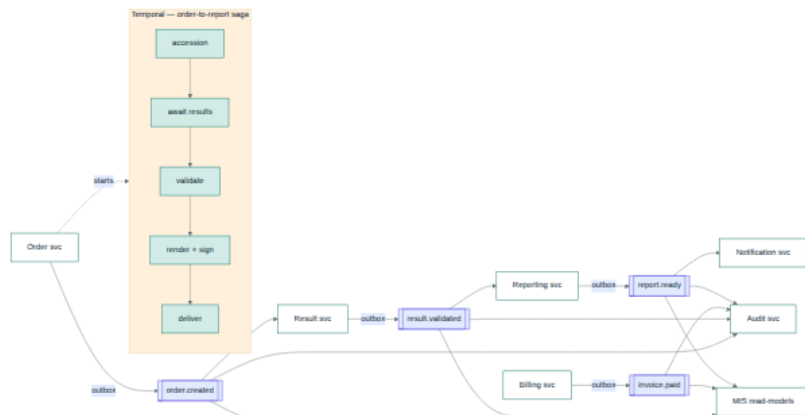


C4-L3 — Order & Workflow (hexagonal)

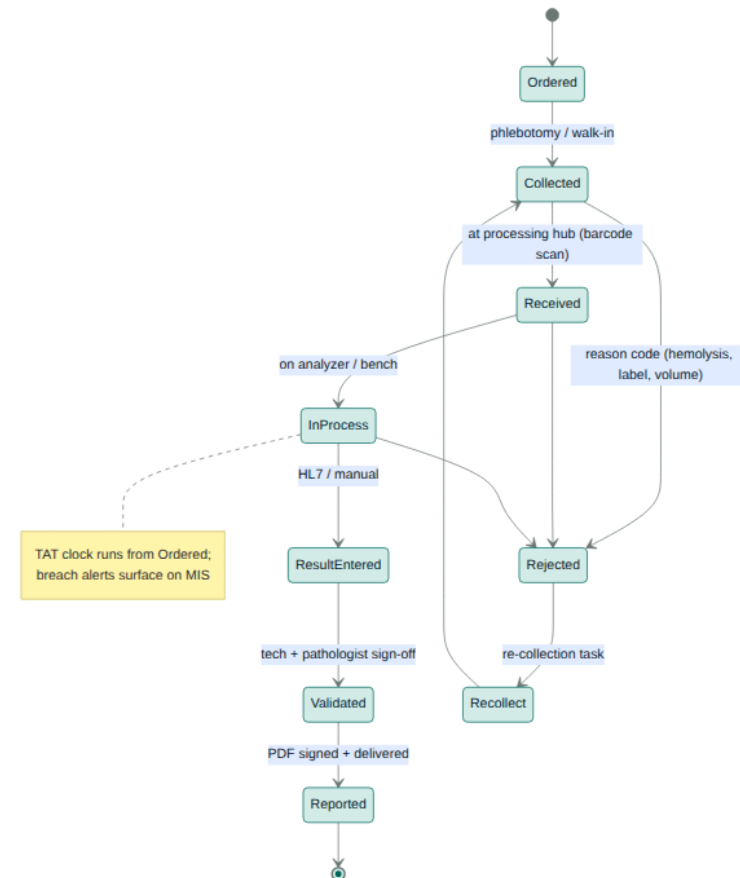


Sequence — register → bill → result → report

Event/saga backbone · Sample state machine

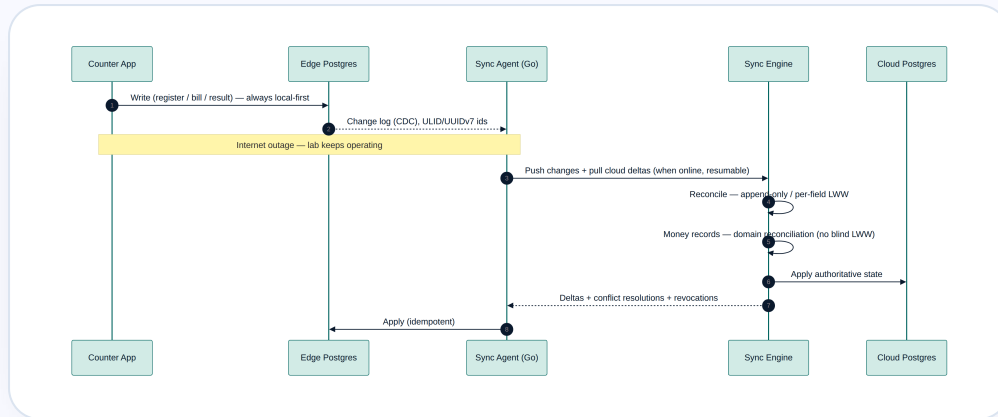


Kafka topics + Temporal order-to-report saga

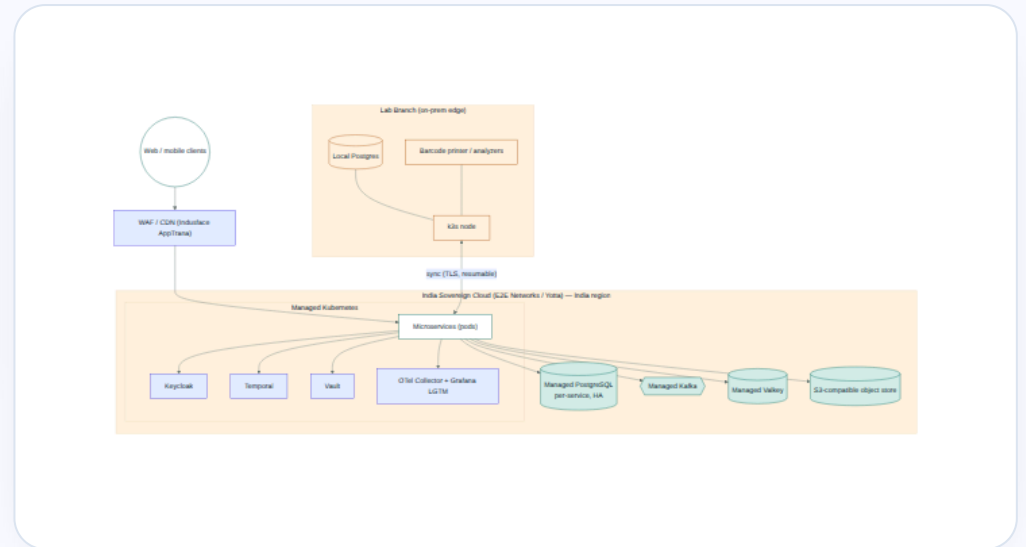


Sample lifecycle & TAT state machine

Offline-first sync · Deployment

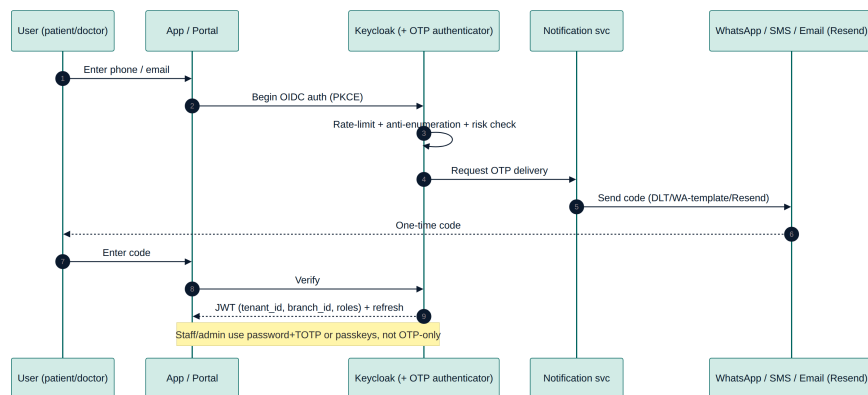


Edge ↔ cloud sync, money reconciliation

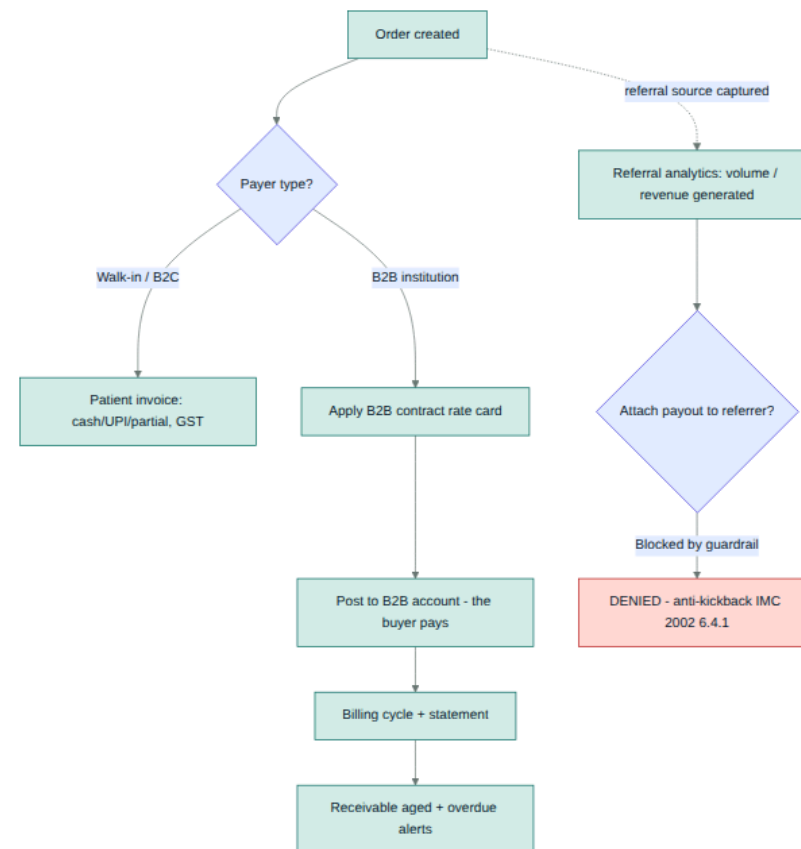


India sovereign cloud + branch edge (k3s)

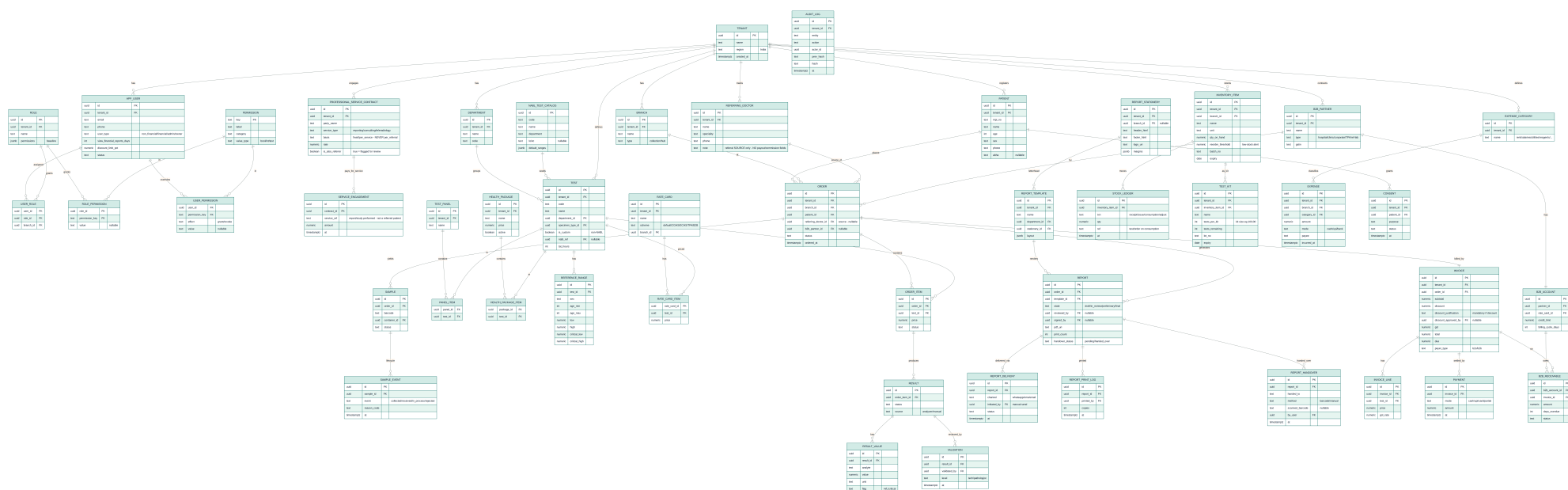
OTP authentication · Compliant B2B billing



Keycloak OTP via WhatsApp/SMS/email



B2B billing — payout-to-referrer blocked (anti-kickback)



Data model — table groups



Tenancy & platform: tenant, branch, app_user (*user_type, limits*), role, user_role, permission, role_permission, user_permission (*owner-defined granular RBAC*), consent, audit_log (hash-chained), notification, idempotency_key, outbox_event.

Master data: department, nabl_test_catalog (*global*), test (*department_id, is_custom, nabl_ref*), test_panel, panel_item, health_package/health_package_item, reference_range, specimen_type, container, rate_card, rate_card_item, report_stationery (*letterhead*), report_template.

Parties: patient (MPI), referring_doctor (*referral source — no payout/commission fields*), b2b_partner, b2b_account, professional_service_contract/service_engagement (*fixed/per_service — never per-referral*).

Orders & samples: order, order_item, sample, sample_event, accession.

Results & reports: result, result_value, validation, report (*review→sign-off, print_count, handover*), report_delivery, report_print_log, report_handover (*barcode*).

Billing & expenses: invoice (*discount_justification*), invoice_line, payment, b2b_receivable, inventory_item, test_kit (*tests_per_kit*), stock_ledger, expense_category/expense.

Conventions: UUIDv7/ULID PKs · tenant_id + RLS on every tenant table · timestamptz · numeric(12,2) money · JSONB for FHIR/flexible · FK + tenant-leading composite indexes.

V1/V2 (described): qc_run, lj_point, supplier, purchase_order, appointment, home_collection_visit, phlebotomist, abha_link, fhir_resource, nhcx_claim, study, pcprdt_form_f.

Compliance in the schema: there are **no referral-commission/payout tables or columns anywhere**; referral sources are tracked for analytics only, a guardrail blocks attaching a payout to a referrer, and the only money-to-a-professional path is a **service contract** (fixed/per_service, never per-referral) — ADR-007 & ADR-010.

Millions of records & high txn/min

↗ Write throughput

- **Citus sharding** by tenant_id (co-located)
- **Monthly partitioning** of append tables
- **Kafka buffer** + batched inserts (COPY)
- Outbox + idempotent consumers
- **UUIDv7** = append-friendly indexes

📡 Read & latency

- **Read replicas** for queries/reporting
- **CQRS read models** for dashboards
- **ClickHouse** for analytics at scale
- **Redis/Valkey** cache hot lookups
- OpenSearch for search

⚙ Ops & safety

- **PgBouncer** transaction pooling
- tenant-leading + partial + BRIN indexes
- HPA + Kafka-lag autoscaling
- Backpressure + DLQ + per-tenant limits
- Hot→warm→cold partition archival (DPDP)

Day-1 vs scale-out: MVP runs lean (one managed Postgres/service + PgBouncer + partitioning + cache + Kafka + a read replica). When metrics demand, add Citus shards, replicas and ClickHouse — **no schema rewrite**, because tenant_id, UUIDv7 PKs and time partition keys are baked in. See scalability-and-data-at-scale.md.

Capacity planning & load testing



Sized bottom-up: op-mix per busy lab (~17k write-rows/day) → peak write rate ≈ 1.3 writes/sec per active lab (8 active hrs, 28% in the peak hour). Reads $\approx 5\times$, events $\approx$ writes. So **peak write TPS $\approx 1.3 \times$ active labs.**

~78k

writes/min
1,000 labs · ~1.3k TPS

~400k

writes/min
5,000 labs · ~6.6k TPS

~1.0M

writes/min
~13,000 labs · ~17k TPS

~2.0M

writes/min
~25,000 labs · ~33k TPS

"Millions of transactions per minute" $\approx$ a 13k–25k busy-lab network — reached by adding nodes along axes the schema already supports (tenant_id shard key, UUIDv7 PKs, monthly partitions). No rewrite.

Provisioning ladder

- **T1 Pilot** ≤ 100 labs — 1 managed PG/svc, 1 replica, 3 Kafka brokers, Redis HA
- **T2 Growth** ~1k — Citus 2–3 workers, 2 replicas, 24 part, ClickHouse 1
- **T3 Scale** ~5k — Citus coord + 4–8 workers, HPA 6–10, 48–96 part, CH 3
- **T4 Hyperscale** 13k–25k — Citus 16–32 workers, autoscale 16–40+, 192+ part, CH 6+

k6 profiles & SLO gates

- **Smoke** · **Load** (hold peak) · **Stress** (find the knee)
- **Soak** (leaks/lag) · **Spike** (autoscale) · **Breakpoint** (capacity → next tier)
- Gates: counter **p95<1s** · OLTP write **p95<50ms / p99<150ms**
- worklist p95<100ms · MIS p95<500ms · errors<1% · checks>99%

CI & method: sized by Little's Law (in-flight = TPS $\times$ service-time) + PgBouncer connection math; nightly Load + weekly Soak in GitHub Actions vs staging; Breakpoint before each tier promotion (safe ceiling = breach-TPS $\times$ 0.7). Plus a Kafka-lag async test and a Toxiproxy offline-sync chaos test. See capacity-and-load-testing.md.